

AUTOMATED SEISMIC DATA PROCESSING

First Break Picking

Developing an automated algorithm to detect first-break arrival times in seismic traces

Team name : Teenage mutant ninja turtles

Project Team

Brunswick · Halfmile · Lalor · Sudbury seismic assets

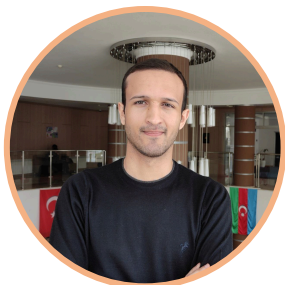
Team Contributions



Ferman Khankishiyev

Team Lead

CI
Deploy
Model Factory
Project Author
Individual Models
Model Test / Training
Unet / LightUnet Implementation
Etc



Bayram Bayramov

Developer Lead

Sweep
Model Pairs
Helper Scripts
Batch Training Core
ONNX / Torchscript
Batch Training Recovery
Preprocessing Foundation
Etc



Mahmud Ramazanov

Developer & Researcher

Docstring
Readme.md
Data Pipeline
EDA Datasets
Preprocessing
Preprocessing Scripts
Configuration System / Foundation
Etc



Murad Taghizade

Documentation Lead
Developer & Researcher

Metrics
Train Scripts
Loss Functions
Documentation
Callbacks / Trainer
MLFlow / TensorBoard
Trainer Advanced Features
Etc



Avaz Huseynov

Developer & Researcher

Logger
Evaluation
Prediction
Visualization
Memory Utils
Export Model
EDA Datasets
Etc

What is a first break, and why does it matter?

A “first break” is the arrival time of the initial seismic wave at a receiver. This time is the key input for building velocity models, computing static corrections, and depth-time conversions in seismic refraction and tomography studies.

1

Traditional approach

Manual picking by geophysicists across millions of traces — a time-consuming process

2

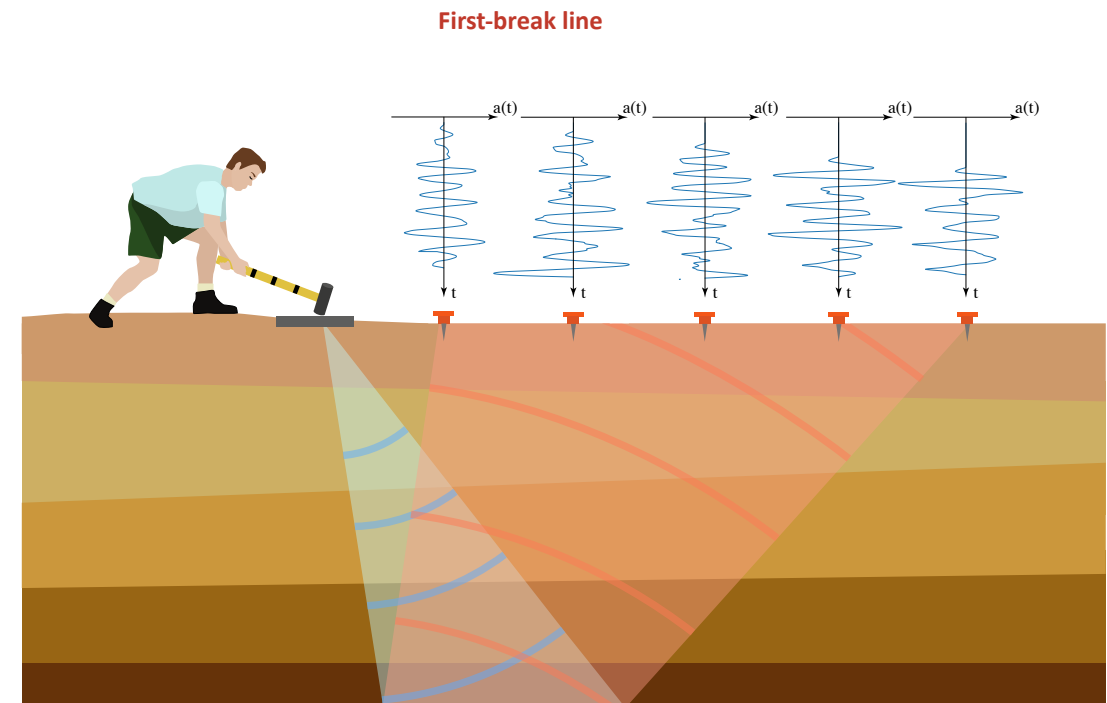
Objective

Build an automated method that performs reliably across varying assets and signal-to-noise ratios

3

Input data

Traces from 4 real seismic assets + expert-labeled first-break times (SPARE1)



Why does this need to be automated?



Time cost

Manually picking millions of traces takes weeks to months and delays project timelines.



Subjectivity

Inconsistency between different pickers' results affects the quality of the velocity model.



Scalability

Manual effort doesn't scale as new assets (Brunswick, Halfmile, Lalor, Sudbury) are added.



Value chain

Faster, more accurate first breaks lead to faster statics and faster exploration decisions.

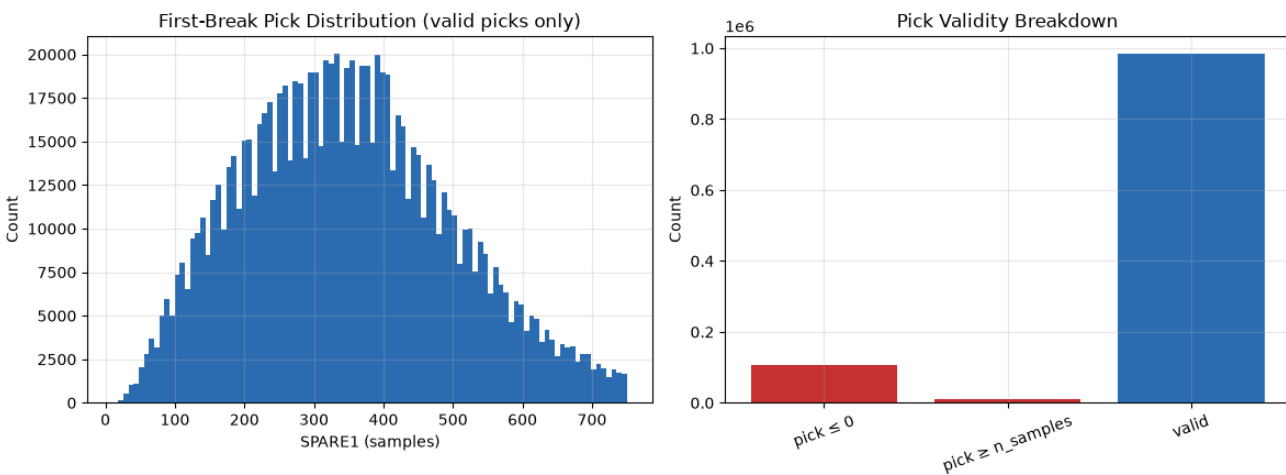
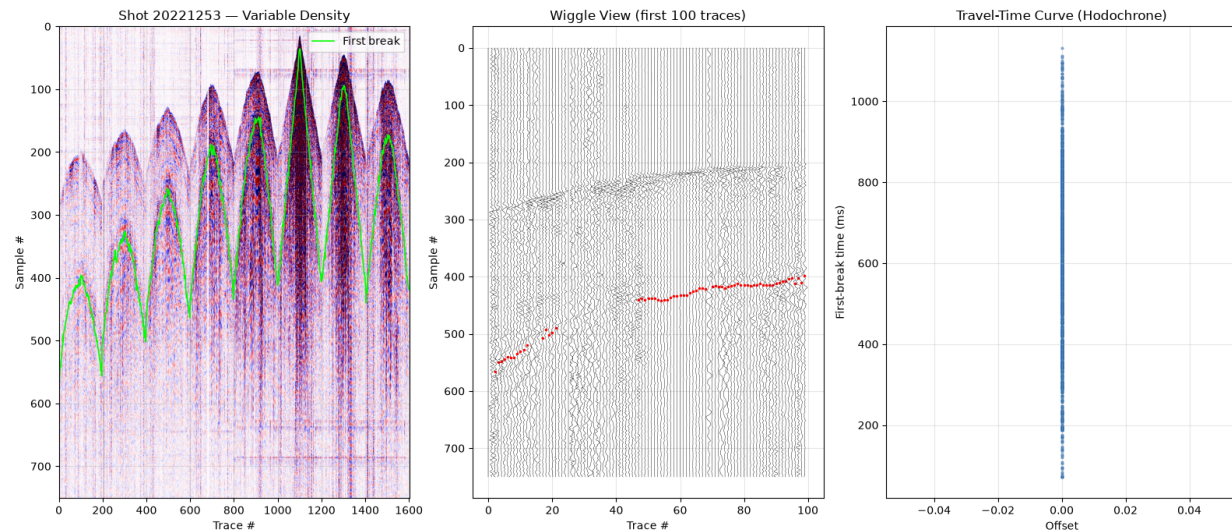
Overview of the four real seismic assets

Brunswick 3D seismic, HDF5 (Brunswick_orig_1500ms_V2)	Halfmile 3D seismic, with added geometry (add_geom_sorted)	Lalor 3D seismic, raw z-values (raw_z_1500ms_norp_geom)	Sudbury 3D seismic, pre-processed
---	--	---	---

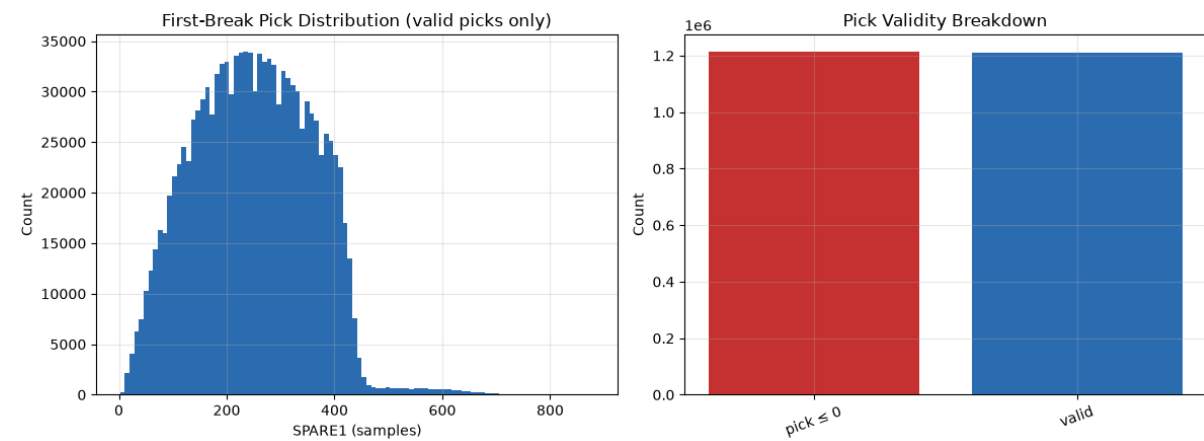
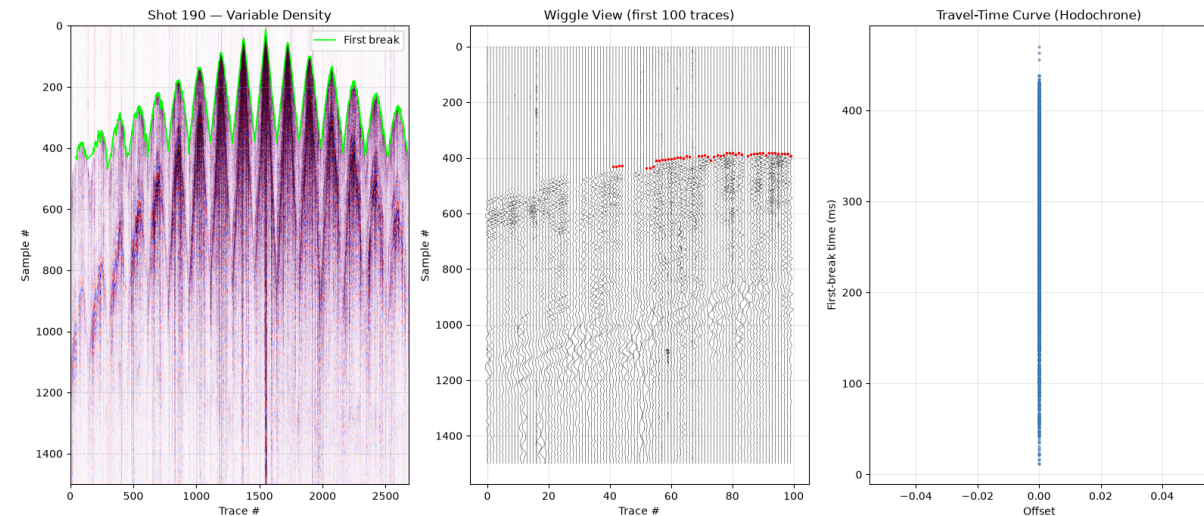
HDF5 structure · /TRACE_DATA/DEFAULT

data_array 2D array of all recorded traces (trace count × sample count)	SOURCE_X / Y / HT Shot (source) coordinates and elevation	REC_X / Y / HT / PEG Receiver coordinates and identifiers
SAMP_RATE / SAMP_NUM Sampling rate (μs) and sample count	SPARE1 First-break time (ms); 0 or -1 = unlabeled	SHOTID / SHOT_PEG Shot identifiers — used to split gathers

Halfmile Dataset and Distribution



Lalor Dataset and Distribution



Data preparation: splitting into gathers

Each HDF5 file stores all traces sequentially. To reconstruct a real 2D seismic image (shot gather), traces are grouped by receiver/source coordinates (SOURCE_X/Y, REC_X/Y) and SHOTID, then each gather is paired with its first-break labels (SPARE1).

01

Load raw data

Load data_array and metadata fields with h5py

02

Split into gathers

Group traces by SHOTID / coordinates

03

Align labels

High-millisecond picks are marked invalid.

04

Clean & filter

Detect unlabeled traces for masking in the loss function

05

Train / Val / Test split

We only implement in 80/10/10 Format.

Software and tooling stack

Data processing

- Python, h5py
- NumPy / Pandas
- SciPy (signal filtering)
- HDF5 seismic datasets
- Trace & gather preprocessing
- Label alignment & validation

Modeling

- PyTorch / TensorFlow
- U-Net architecture
- Light / Tiny / Nano U-Net variants
- MobileUNet architecture
- MPS-based training
- MLflow experiment tracking

Visualization & QC

- Matplotlib
- Interactive gather viewer
- Jupyter notebooks
- TensorBoard
- Training / validation curves
- Prediction & first-break QC plots notebooks

Implemented models - strengths and weaknesses

Model	Strengths	Weaknesses
U-Net (Best)	Full encoder-decoder depth gives the highest segmentation accuracy and the most precise boundary localization.	Heaviest model - highest memory and compute cost, slowest inference.
Tiny U-Net	Good balance between accuracy and computational efficiency.	Noticeably less accurate than the full U-Net.
Pico U-Net (Worst)	Extremely fast inference with a minimal memory footprint.	Lowest accuracy and reliability of all variants - too small to capture fine detail.
Nano U-Net	Very low compute requirements while still detecting first-breaks effectively.	Ultra-compact design limits accuracy on complex or noisy data.
Mobile-Net	Optimized for fast, resource-efficient detection on mobile hardware.	Mobile-oriented design trades off some segmentation precision.
MPS Light U-Net	Optimized specifically for efficient inference on Apple Silicon (MPS).	Performance gains are platform-specific; less advantage on non-Apple hardware.
Light U-Net	Reduced computational cost while keeping efficient segmentation.	Lighter architecture sacrifices some accuracy versus the full U-Net.

Metrics

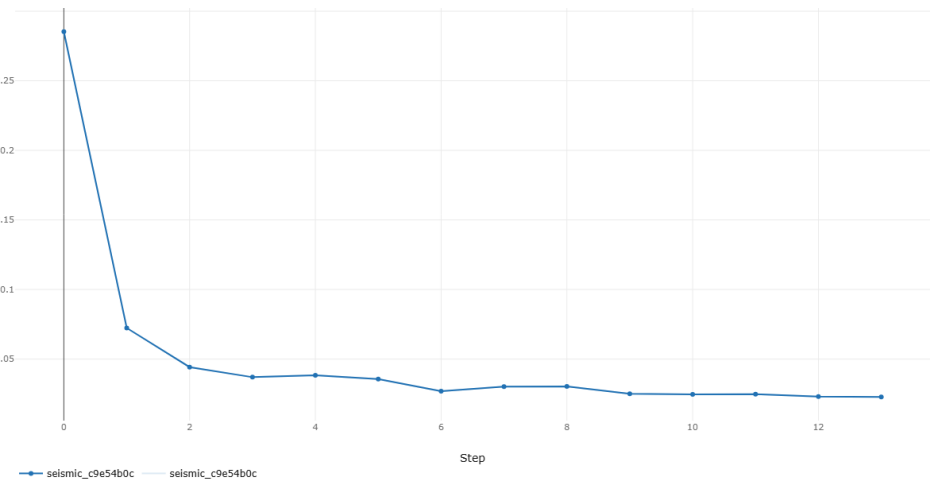
Model output is compared against the expert-labeled SPARE1 times. The metrics below are computed per asset and in aggregate across all assets.

Model	System	First Break Picking	Data
IoU,F1,Accuracy,Recall,Precision,Train/Val Loss	Memory Usage &Performance	Min, Max, Mean, Std absolute error	Ratio, trace/sample count, invalid picks
<i>Core metrics for evaluating segmentation quality and training stability.</i>	Tracks how much memory the model uses and how fast it runs.	Summarizes the spread and average size of the timing error.	<i>Tracks how many picks are valid versus unreliable across traces.</i>

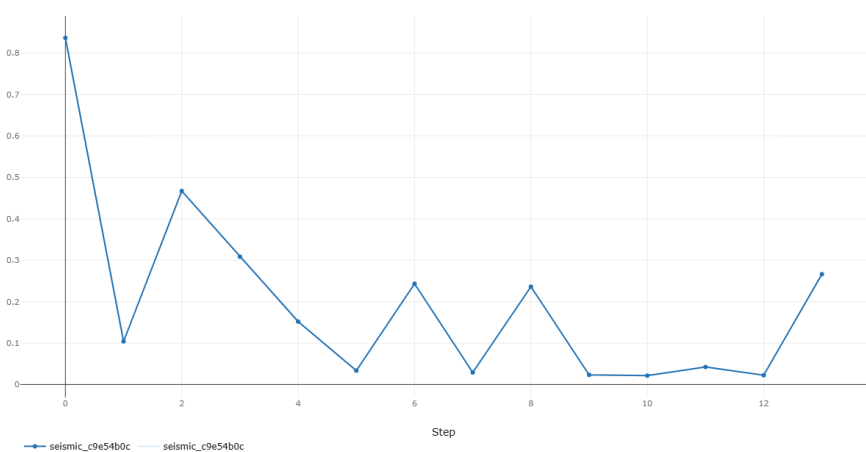
Note: fill in actual MAE and pick-rate figures here once validation results are available.

Part Of Metrics Results

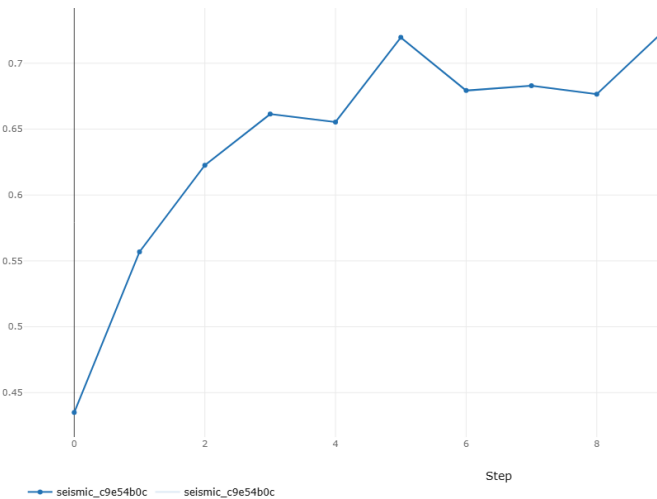
Train Loss U-Net



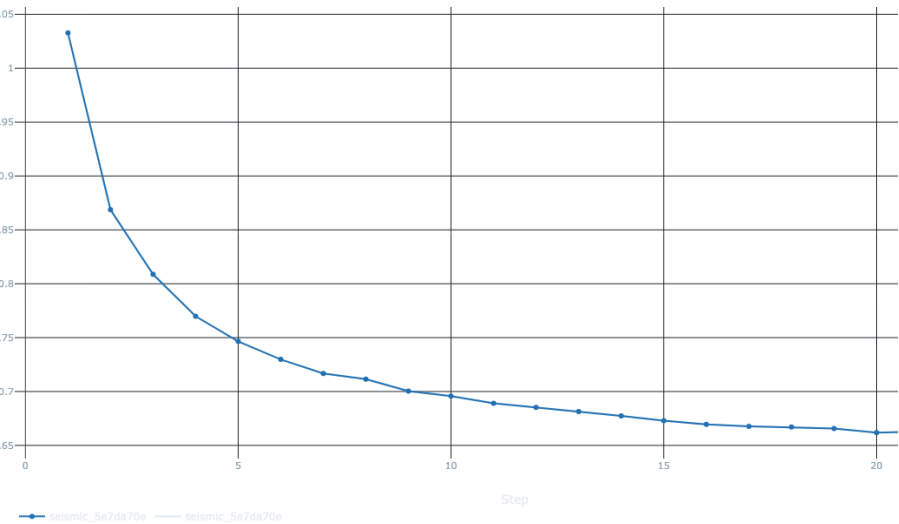
Validation Loss U-Net



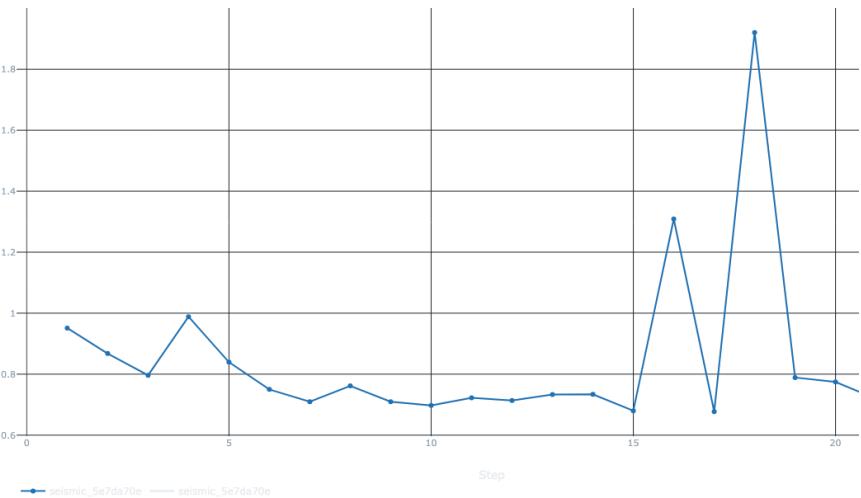
Val IoU Loss U-Net



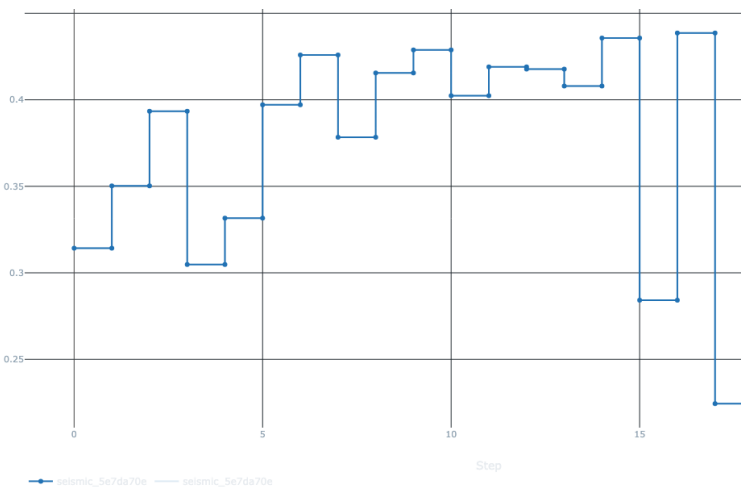
Train Loss Pico U-Net



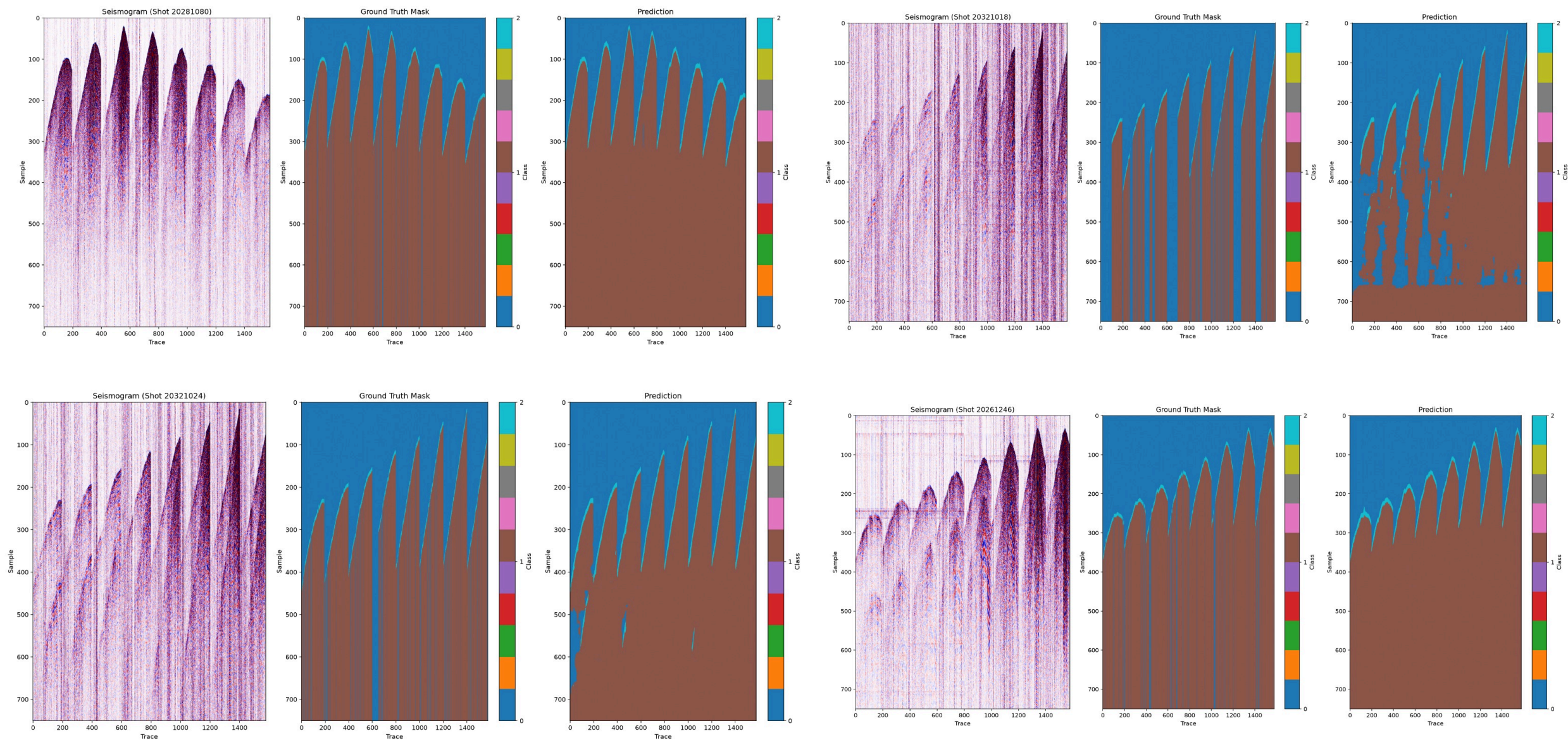
Validation Loss Pico U-Net



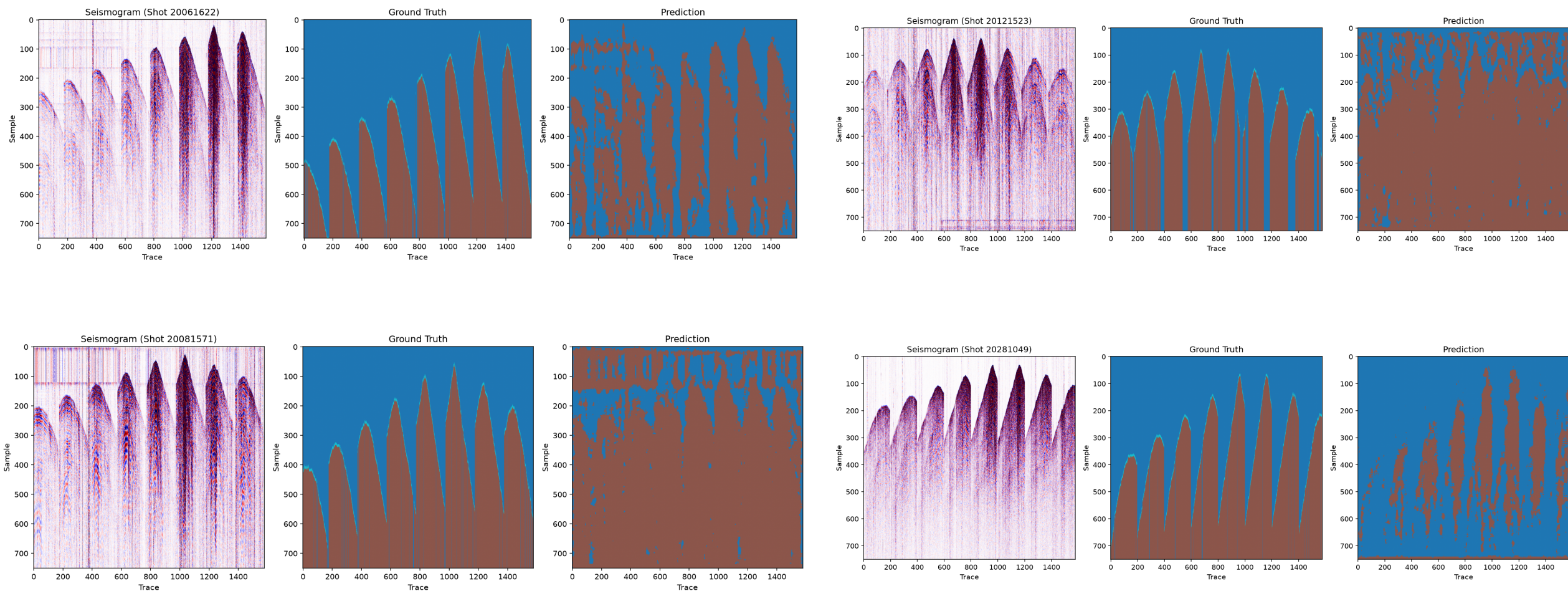
Val IoU Pico U-Net



Prediction Results (U-Net Model Best One HalfMile Dataset)



Prediction Results (Pico U-Net Model Worst One HalfMile Dataset)



Limitations

- **Limited Training Resources:** The required training hardware was not provided, and no dedicated compute resources were allocated for model training.
- **High Memory Consumption:** Our models tend to retain a significant amount of memory, creating deployment and storage challenges, particularly due to the large size of the seismic datasets.
- **CPU Limitations:** The available systems have limited CPU capacity, making frequent CPU access for processing and feeding large datasets into the training pipeline inefficient.
- **GPU Limitations:** GPU resources were also constrained by insufficient VRAM and computational capacity. The provided A100 GPU was migrated to a **42 SM configuration**, which limited the available computing power for large-scale model training.

Our Strengths & Improvements

- **Efficient Model Design:** We developed and evaluated multiple lightweight U-Net-based architectures to balance model performance with limited computational resources. 1D/2D CNN — per-trace time segmentation
- **Multi-Dataset Evaluation:** We worked with four real seismic datasets — **Halfmile, Lalor, Brunswick, and Sudbury** — providing broader evaluation across different data characteristics.
- **Memory-Efficient Training:** We implemented chunk-based processing, optimized data loading, and lightweight configurations to handle large seismic datasets under memory constraints.
- **Multiple Model Comparison:** We evaluated different architectures, including U-Net, Light U-Net, Efficient U-Net, and MobileNet-based models, rather than relying on a single model.
- **Robust Training & Experiment Pipeline:** We developed structured training, evaluation, checkpointing, logging, and experiment-tracking components to make experiments more reproducible and systematic.

Achieved results vs. planned goals

Planned

- A single, automated pipeline across all 4 assets
- Consistent accuracy across varying SNR conditions
- Significantly reduce expert manual effort

Achieved

- Gather-based data preparation pipeline completed
- Baseline and deep-learning models built and compared
- Cross-asset validation results presented

Thank You!

We welcome your questions